

Relatório da Etapa 6 do PI

Refatoração e separação de responsabilidades para reuso web

Aluno: Gustavo C

1. Contexto

A Etapa 6 do Projeto Integrador pede a preparação do código para reutilização em um sistema web: separar as regras de negócio da interface, aplicar SOLID, eliminar code smells e versionar o resultado no GitHub. Como não havia sistema desktop da etapa anterior para refatorar, foi elaborado um novo projeto (Sistema de Gestão de Clínica), conforme previsto no enunciado. O documento de requisitos acompanha este relatório (Documento_de_Projeto.docx).

Para tornar a refatoração demonstrável, a pasta desktop-legado contém um formulário no estilo NetBeans/Swing (FormPacienteLegado.java) que reúne os problemas típicos de telas desktop. Ele representa o "antes"; o projeto SistemaClinicaWeb é o "depois". O repositório da entrega contém ambos.

2. Code smells encontrados e refatorações aplicadas

Code smell	Onde (antes)	Refatoração	Onde (depois)
God Class / Long Method	FormPacienteLegado: jButton1ActionPerformed lê campos, valida, monta SQL e mostra mensagens.	Extract Class e Extract Method	PacienteService, PacienteRepository, validadores
Regra de negócio na interface	Validações de nome, CPF e telefone dentro do actionPerformed.	Move Method para camada de serviço	NomeObrigatorioValidador, CpfValidador, TelefoneValidador, DataNascimentoValidador
Acesso a dados na interface	DriverManager e PreparedStatement dentro da tela.	Extract Interface e Extract Class	PacienteRepository e suas implementações
Código duplicado	replaceAll("\\D", "") e a conexão repetidos nos dois botões.	Extract Method / classe utilitária	CpfUtil.somenteDigitos
Magic strings e números	"AGENDADA", 11, 10, 11 espalhados.	Replace Magic Literal com enum	StatusConsulta; regras concentradas nos validadores
Nomes genéricos	jTextField1, jButton1, variáveis a, b, c.	Rename	Atributos e variáveis com nomes de domínio (nome, cpf, telefone)
Credenciais no código	Usuário e senha do banco escritos na tela.	Isolar a configuração de acesso	Fica restrito à implementação de repositório; a interface nunca vê a conexão

Code smell	Onde (antes)	Refatoração	Onde (depois)
Tratamento de erro misturado à UI	JOptionPane para cada erro.	Replace Error Code with Exception	RegraNegocioException; a interface decide como exibir

2.1 Exemplo: antes

```
private void jButton1ActionPerformed() {
    String a = jTextField1.getText();
    String b = jTextField2.getText().replaceAll("\\D", "");
    if (a == null || a.trim().length() < 3) {
        JOptionPane.showMessageDialog(this, "Nome inválido");
        return;
    }
    ... // validação de CPF e telefone
    Connection con = DriverManager.getConnection("jdbc:mysql://localhost/clinica",
"root", "1234");
    PreparedStatement ps = con.prepareStatement("INSERT INTO paciente ...");
    ...
}
```

2.2 Exemplo: depois

```
public Paciente cadastrar(Paciente paciente) {
    validadores.forEach(v -> v.validar(paciente));
    paciente.setCpf(CpfUtil.somenteDigitos(paciente.getCpf()));
    repositorio.buscarPorCpf(paciente.getCpf()).ifPresent(existente -> {
        throw new RegraNegocioException("Já existe paciente cadastrado com este
CPF.");
    });
    return repositorio.salvar(paciente);
}
```

O método deixou de conhecer Swing e SQL. Uma futura página web (ou a tela Swing) apenas chama `PacienteService.cadastrar` e trata a exceção.

3. Princípios SOLID aplicados

Princípio	Classes afetadas	Justificativa
S: Responsabilidade Única	Paciente, Consulta (dados); PacienteService, ConsultaService (regras); repositórios (persistência); cada Validador (uma regra); CpfUtil (CPF)	A classe da tela acumulava leitura de campos, validação, SQL e mensagens. Cada motivo de mudança agora está em uma classe: alterar a regra do telefone mexe só em TelefoneValidador.
O: Aberto/Fechado	PacienteService, ValidadorPaciente e suas implementações	Para criar uma nova regra basta escrever outro ValidadorPaciente e incluí-lo na lista; PacienteService não é modificado.
L: Substituição de Liskov	PacienteRepositoryArquivo, PacienteRepositoryMemoria	A versão em arquivo estende a versão em memória mantendo o mesmo contrato. O Main executa a mesma bateria de testes nas duas, com resultado idêntico.
I: Segregação de Interfaces	Leitura, Escrita, PacienteRepository, ConsultaRepository, ConsultaService	ConsultaService só precisa consultar pacientes e por isso depende de <code>Leitura<Paciente></code> , sem enxergar salvar ou remover.

Princípio	Classes afetadas	Justificativa
D: Inversão de Dependência	PacienteService, ConsultaService, Main	Os services dependem de interfaces (PacienteRepository, ValidadorPaciente) recebidas no construtor. Somente o Main conhece as classes concretas. Trocar memória por arquivo (ou banco) não altera os services.

4. Padrões de projeto utilizados

Padrão	Onde	Justificativa
Strategy	ValidadorPaciente e as quatro implementações	Cada regra de validação é intercambiável e independente; o serviço percorre a lista sem conhecer as regras.
Repository (DAO)	PacienteRepository, ConsultaRepository e implementações	Isola o acesso a dados; permite trocar a fonte de dados e testar sem banco.
Injeção de dependência (via construtor)	PacienteService, ConsultaService, Main	Concretiza a inversão de dependência, sem framework.

5. Testes no método main()

Conforme o enunciado, os testes foram implementados no main() (classe Main). Sem JUnit, cada verificação imprime [OK] ou [FALHOU], e o programa termina com código de saída 1 se algum teste falhar. Foram executados 31 testes, todos aprovados, cobrindo:

- Cadastro válido, geração de id e normalização do CPF.
- Rejeição de CPF inválido, nome curto, data de nascimento futura, telefone curto e CPF duplicado.
- Atualização, remoção e listagem de pacientes.
- Agendamento, conflito de horário, data passada, paciente inexistente, especialidade vazia e cancelamento.
- A mesma bateria de pacientes com o repositório em arquivo e a persistência após reabrir o arquivo.

Trecho da saída da execução:

```
=== Testes com repositório em MEMÓRIA ===
[OK]    Cadastro válido gera id
[OK]    CPF inválido é rejeitado -> "CPF inválido."
...
[OK]    Dados persistem após reabrir o arquivo
```

Resultado: 31 passaram, 0 falharam.

Para executar: no NetBeans, abrir o projeto (File > Open Project) e usar Run Project, ou pelo terminal, `mvn compile exec:java -Dexec.mainClass=br.com.clinica.Main`.

6. Evidências do repositório GitHub

Link do repositório: [cole aqui a URL do seu repositório]

Inclua neste relatório as capturas de tela abaixo (substitua os espaços reservados):

- [Print 1] Página inicial do repositório, com a lista de pastas e o README.
- [Print 2] Histórico de commits (aba Commits), mostrando a sequência de refatoração.
- [Print 3] Estrutura de pacotes do projeto SistemaClinicaWeb no GitHub.
- [Print 4] Saída do Main no NetBeans com os 31 testes aprovados.

6.1 Sugestão de sequência de commits

#	Mensagem de commit
1	Adiciona código desktop legado (FormPacienteLegado) como ponto de partida
2	Cria projeto Maven SistemaClinicaWeb e entidades do modelo
3	Extraí validações para classes Strategy (validation)
4	Extraí acesso a dados para interfaces de repositório e implementação em memória
5	Cria PacienteService e ConsultaService com injeção de dependência
6	Adiciona repositório em arquivo (Liskov) e testes no Main
7	Adiciona documento de projeto, relatório e README

7. Conclusão

A separação em camadas removeu do código toda dependência de interface gráfica: as classes de model, validation, repository e service podem ser usadas sem alteração por uma aplicação web. A aplicação dos princípios SOLID também simplificou o teste do sistema, já que os services funcionam com repositório em memória e o mesmo código rodou com persistência em arquivo. Nas próximas etapas, uma camada web e um repositório com banco de dados poderão ser acrescentados sem alterar as regras de negócio.